Construcción del grafo, paso a paso

Fases 2 y 3 de `gtfs-ingest` sobre un modelo de juguete: dos feeds GTFS, un CSV de MiBici y 13 paradas

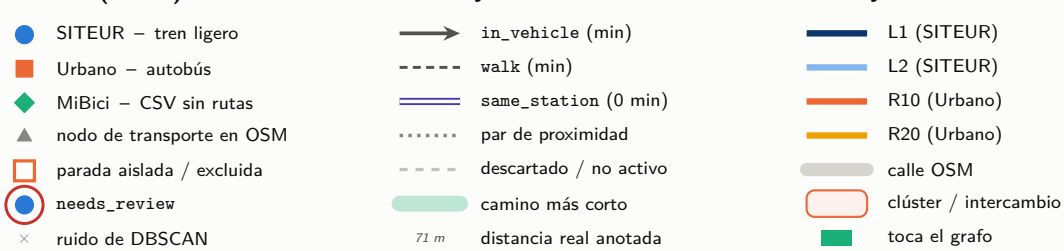
Víctor Rodríguez

16 de septiembre de 2026

Deduplicación mostrada: regla clásica entre feeds (30 m + nombre o ruta compartida). Los números del modelo son ilustrativos; las fórmulas y reglas son las del código.

Simbología

La misma en todas las diapositivas

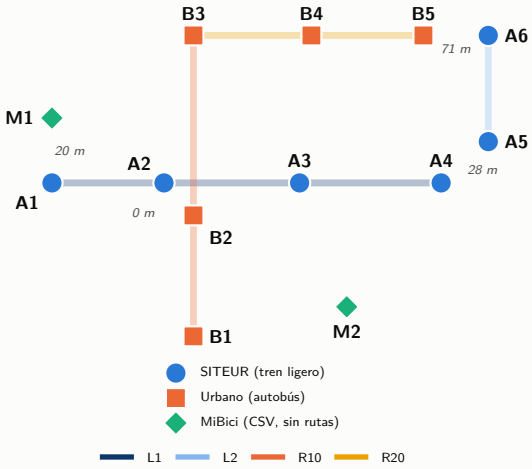


Los mapas son esquemáticos (no a escala): la distancia que usan los algoritmos va anotada en cursiva.

El modelo de juguete

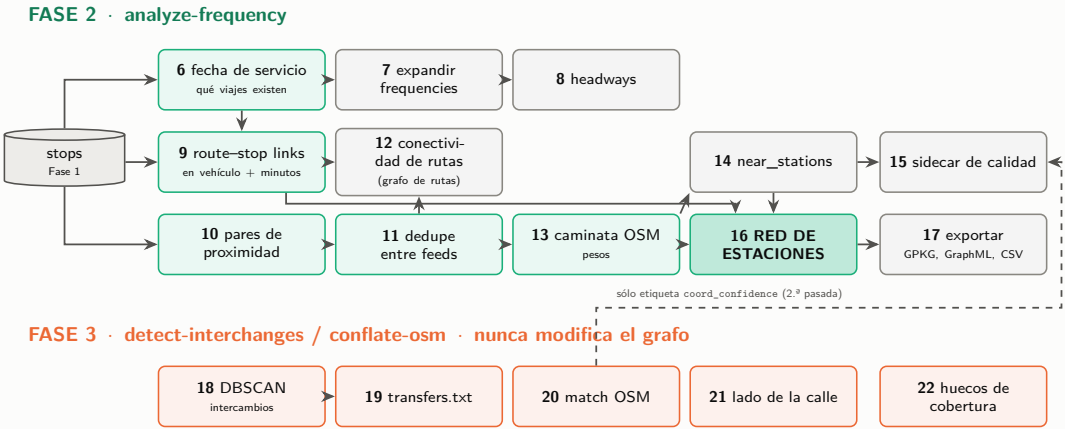
Tres fuentes, cinco rutas, 13 paradas · fecha de análisis: miércoles

`siteur` – GTFS, tren ligero (`route_type 0`)	
Rutas	L1: A1→A2→A3→A4 L2: A5→A6
Estación	A4 y A5 comparten `parent_station` “Centro”
transfers.txt	A4 ↔ A5
`urbano` – GTFS, autobús (`route_type 3`)	
Rutas	R10: B1→B2→B3 (viajes explícitos)
	R20: B3→B4→B5 (`frequencies.txt`, cada 10 min)
	R30: B3→B4 (sólo sábado)
Nombres	A2 = “Juárez”, B2 = “JUAREZ”
transfers.txt	B4 ↔ B5
`mibici` – CSV `id, name, latitude, longitude`	
Paradas	M1 “Ciclopuerto”, M2 “GDL-200” (sin rutas)



Del dato al grafo: qué pasos lo tocan

Verde = modifica nodos, aristas o pesos del grafo · gris = sólo otros artefactos

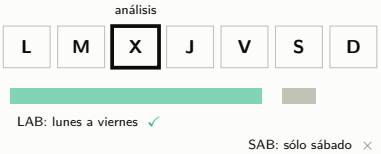


Fase 2

`analyze-frequency`: el grafo se construye aquí

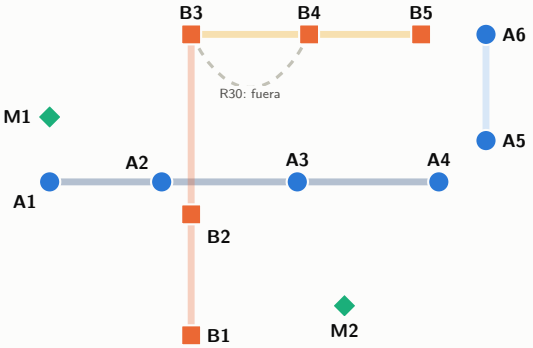
6 fecha de servicio **7** frecuencias **8** headways **9** enlaces de ruta
10 proximidad **11** dedupe **12** conectividad **13** caminata OSM
14 near_stations **15** calidad **16** red de estaciones **17** exportar

PASO 6 Fecha de servicio: qué viajes existen el miércoles



Ruta	service_id	¿Activa?
L1, L2	LAB	✓
R10, R20	LAB	✓
R30	SAB	×

$activos(d) = \{\text{bandera del día} \wedge inicio \leq d \leq fin\} \cup \text{agregados} \setminus \text{quitados}$

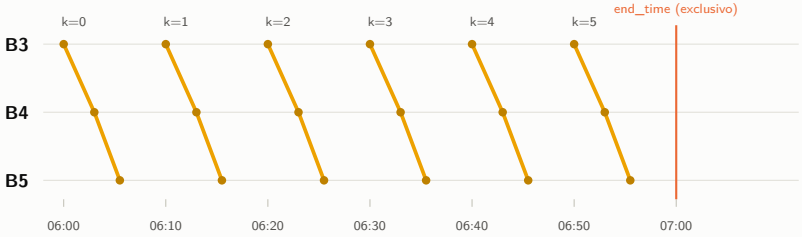


Efecto en el grafo

El filtro cae en cascada: viajes → `stop_times` → rutas. R30 no genera ninguna arista `in_vehicle`. Las paradas B3 y B4 siguen siendo nodos (las sirven R10 y R20).

PASO 7 Expandir `frequencies.txt` (R20)

Una plantilla de viaje se convierte en 6 corridas reales — sólo para headways



Plantilla T20
B3 +0:00
B4 +3:00
B5 +5:30

Regla

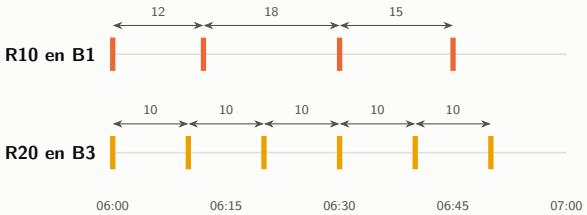
$n = \min\left(\lceil (07{:}00 - 06{:}00)/10\,\text{min} \rceil, 1000\right) = 6$
$\text{salida}_k = 06{:}00 + k \cdot 10$ min + desfase de la parada
las filas de la plantilla se **reemplazan**, nunca se duplican

No toca el grafo

Las 6 corridas visitan B3→B4→B5 con los mismos desfases: el orden de paradas y `travel_min` (3.0, 2.5) salen idénticos de la plantilla. Por eso `travel_n_obs` $= 1$.

PASO 8 Headways por parada, ruta, dirección y ventana

Ventana am_peak 06:00–09:00



	R10	R20
salidas N	4	6
huecos $N-1$	3	5
media (min)	15	10
mediana	15	10
mín / máx	12 / 18	10 / 10

Salidas

`route_stop_headways`: una fila por parada/ruta/dirección/ventana
`route_headways`: media sin ponderar de las medias por parada

No toca el grafo

R10 (cada 15 min) y R20 (cada 10 min) quedan como aristas iguales en el grafo: no existe un costo de espera en `station_network`.

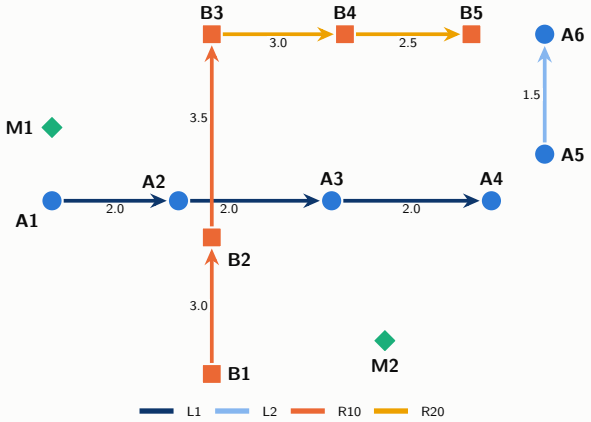
PASO 9 Enlaces ruta–parada: las aristas en vehículo

Paradas consecutivas según `stop_sequence`; minutos = mediana entre viajes

Ruta	De	A	travel_min	n_obs
L1	A1	A2	2.0	40
L1	A2	A3	2.0	40
L1	A3	A4	2.0	40
L2	A5	A6	1.5	40
R10	B1	B2	3.0	4
R10	B2	B3	3.5	4
R20	B3	B4	3.0	1
R20	B4	B5	2.5	1

Regla

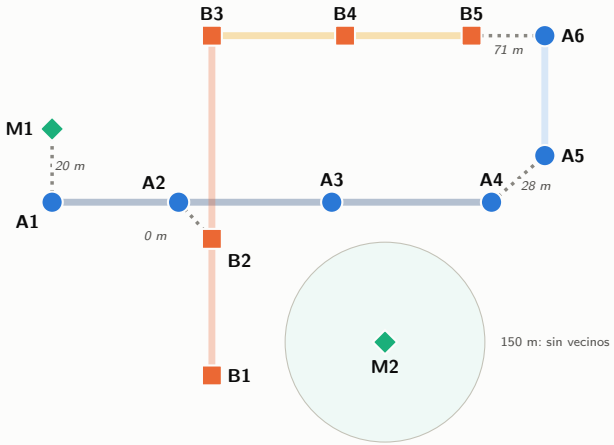
seg = salida$_{i+1}$ − salida$_i$ (negativo → NULL)
agrupar (ruta, dirección, parada, siguiente)
travel_min = $\mathrm{mediana}(seg)/60$ *n_obs* = viajes con tiempo



8 arcos dirigidos. Por claridad sólo se dibuja la dirección 0. M1 y M2 no tienen rutas: nodos sin arcos en vehículo.

PASO 10 Pares de proximidad (`--proximity-m 150`)

Todo par de paradas a ≤ 150 m, de cualquier fuente



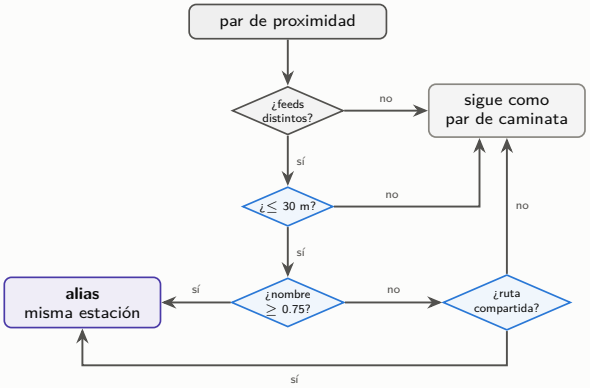
Par	Fuentes	m
A1 – M1	siteur / mibici	20
A2 – B2	siteur / urbano	0
A4 – A5	siteur / siteur	28
A6 – B5	siteur / urbano	71

$$pares = \{(a, b) : uid_a < uid_b,\ d(a, b) \leq 150\}$$

- PostGIS: `ST_DWithin` sobre `geography`, con índice GiST
- M2 no tiene ningún par: será un nodo **aislado**
- Estos 4 pares alimentan dedupe (11), conectividad (12) y caminatas (13)

PASO 11 Deduplicación entre feeds

Regla clásica: feeds distintos, $\leq$ 30 m y (nombre $\geq$ 0.75 **o** ruta compartida)

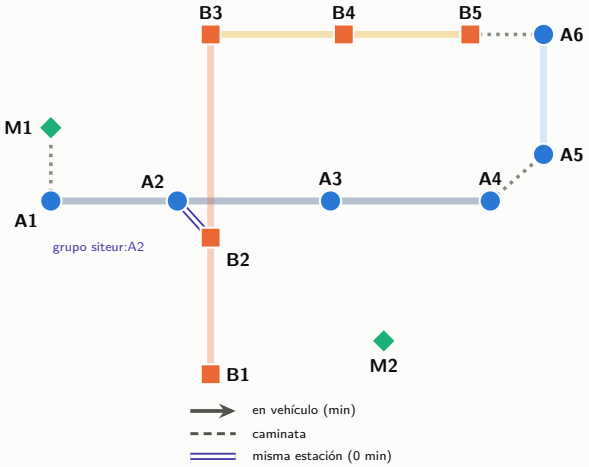


Par	feeds	m	nombre	rutas
A2 – B2	$\neq$	0	1.00	0
$\Rightarrow$ **alias** “juarez” = “juarez”				
A1 – M1	$\neq$	20	$\approx$0.2	0
$\Rightarrow$ rechazado: MiBici no tiene rutas				
A4 – A5	=	28	–	–
$\Rightarrow$ no evaluado: mismo feed				
A6 – B5	$\neq$	71	–	–
$\Rightarrow$ rechazado: más de 30 m				

Resultado

grupo {A2, B2} · `canonical_uid` = `siteur:A2` (el menor)
`score` $= 0.5(1 - 0/30) + 0.5 \cdot 1.0 = 1.0$
`filter_stop_pairs` quita A2–B2 de los pares: **quedan 3**

PASO 11 Efecto de la deduplicación en el grafo



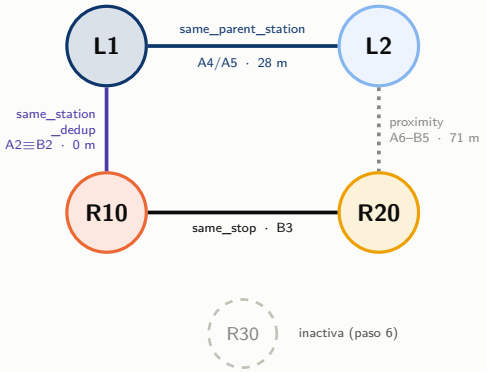
- Nada se borra: las 13 paradas siguen siendo nodos
- A2 y B2 no "caminan" entre sí: los une una arista `same_station` de **0 min**
- Así una persona pasa de L1 a R10 sin costo
- A1–M1 queda como caminata normal: una estación de bicicletas nunca se fusiona con transporte

Contar estaciones

13 registros → **12 estaciones** físicas:
`COUNT(DISTINCT COALESCE(station_dedup, stop_uid))`

PASO 12 Conectividad de rutas: el otro grafo

`route_connectivity.graphml`: nodos = rutas activas, arista = evidencia más fuerte

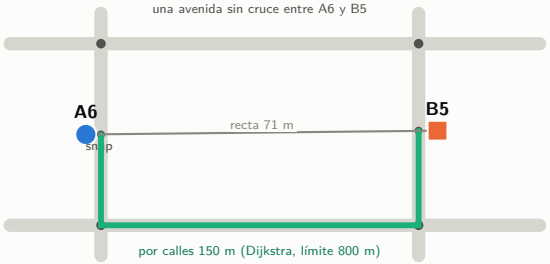


Tipo (fuerza ↓)	Evidencia en el modelo	Par
`same_stop`	R10 y R20 paran en B3	R10–R20
`same_parent_station`	A4 y A5 bajo “Centro”	L1–L2
`same_station_dedup`	alias A2 ≡ B2 (paso 11)	L1–R10
`proximity`	A6–B5 a 71 m	L2–R20

- A4–A5 también es par de proximidad, pero gana `same_parent_station`
- M1 está cerca de A1 pero no tiene rutas: no crea conexión
- Usa distancias en línea recta: la caminata OSM (paso 13) se aplica **después**
- **No** es la red de estaciones: no tiene paradas ni minutos

PASO 13 Caminata sobre calles OSM (`--network-walk`)

Cambia los pesos de las caminatas, no qué pares existen



Par	recta	red	min	walk_source
A1 – M1	20	–	1	straight_line
A4 – A5	28	35	1	network
A6 – B5	71	150	2	network

- A1–M1: no hay camino por calles en 800 m (un canal los separa) ⇒ se conserva la recta
- Sin `--network-walk`, A6–B5 valdría 1 min en vez de 2
- El grafo de calles se construye una vez y queda en caché (`.npz`)

Regla

$red = ruta + snap_a + snap_b \qquad d = \text{máx}(red, recta)$
$min = \text{máx}(1, \text{round}(d/83))$

PASO 14 near_stations y same_line_stations

Dos listas por parada en `near_stations.csv` (y en la capa `stops` del GeoPackage)

ID	stop_uid	**near_stations** (ID, nombre, línea, min)	**same_line_stations** (ID, nombre, línea, dir, min)
1	siteur:A1	(12, Ciclopuerto, -, 1)	(2, Juárez, L1, 0, 2.0)
2	siteur:A2	vacío: B2 es alias, no vecino	(1, Mezquitán, L1, 0, 2.0) (3, Catedral, L1, 0, 2.0)
4	siteur:A4	(5, Centro andén 2, L2, 1)	(3, Catedral, L1, 0, 2.0)
6	siteur:A6	(11, Plaza San Juan, R20, 2)	(5, Centro andén 2, L2, 0, 1.5)
9	urbano:B3	vacío	(8, JUAREZ, R10, 0, 3.5) (10, Mercado, R20, 0, 3.0)
12	mibici:M1	(1, Mezquitán, L1, 1)	vacío: sin rutas
13	mibici:M2	vacío: aislada	vacío

near_stations

Sólo distancia (cualquier línea), minutos del paso 13.
Un vecino sin rutas aparece con línea - (M1 para A1).

same_line_stations

Sólo secuencia (anterior y siguiente, sin límite de distancia).
Minutos = `travel_min` del paso 9; si falta, estimado a 25 km/h.

PASO 15 Sidecar de calidad: cómo se obtuvo cada valor

`near_stations_quality.csv` — mismas filas e IDs; sólo etiquetas

ID	stop_uid	walk_source	travel_source	n_obs	coord_confidence	station_dedup
1	siteur:A1	straight_line	observed	40	0.86	
2	siteur:A2	none	observed	40	0.84	siteur:A2
4	siteur:A4	network	observed	40	needs_review	
6	siteur:A6	network	observed	40	0.80	
8	urbano:B2	none	observed	4	0.84	siteur:A2
10	urbano:B4	none	observed	1	0.90	
12	mibici:M1	straight_line	none	0	needs_review	
13	mibici:M2	none	none	0	needs_review	

walk_source

etiqueta de los vecinos de la parada (paso 13); `none` si no tiene vecinos

travel_source, n_obs

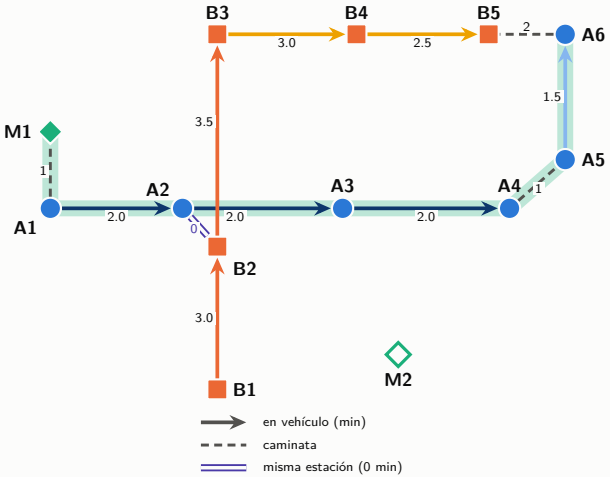
B4 = 1: R20 es una plantilla de `frequencies.txt` (paso 7)

coord_confidence

viene de la Fase 3 (paso 20) en la segunda pasada; no mueve ni borra paradas

PASO 16 La red de estaciones

`station_network.graphml` — todo junto en un solo grafo dirigido



Elemento	Cant.	Origen
nodos (paradas)	13	Fase 1
arcos `in_vehicle`	8	pasos 6, 9
arcos `walk` (3 pares)	6	pasos 10, 11, 13
arcos `same_station` (1 par)	2	paso 11
componentes débiles	2	M2 aislada

Camino más corto M1 → A6

$$\text{M1} \xrightarrow{1} \text{A1} \xrightarrow{2+2+2} \text{A4} \xrightarrow{1} \text{A5} \xrightarrow{1.5} \text{A6} \;=\; \mathbf{9.5\ min}$$

por R10/R20: $1 + 2 + 0 + 3.5 + 3 + 2.5 + 2 = 14$ min

```
nx.shortest_path(G, "mibici:M1", "siteur:A6",
weight="time_min")
```

PASO 17 Exportar: artefactos del modelo

Verde = representación del grafo

`station_network.graphml`

13 nodos · 16 arcos

`transit_analysis.gpkg`

stops 13 · near_links 3
station_network 12 · +2 capas

`station_network_edges.csv`

una fila por par (walk, same_station) y por sentido (in_vehicle)

`route_connectivity.graphml`

4 rutas · 4 conexiones

`near_stations(_quality).csv`

13 filas cada uno

`stop_equivalences.csv`

1 par: A2 – B2

`route_*headways.csv`

R10, R20 por parada y ventana

`analysis_report.json`

fechas, conteos, conectividad

`tablas PostGIS`

headways; conexiones y equivalencias recargadas

La capa `station_network` tiene 12 filas: 8 `in_vehicle` (una por sentido) + 3 `walk` + 1 `same_station` (una por par).

Fase 3

`detect-interchanges` y `conflate-osm`: auditoría, no construcción

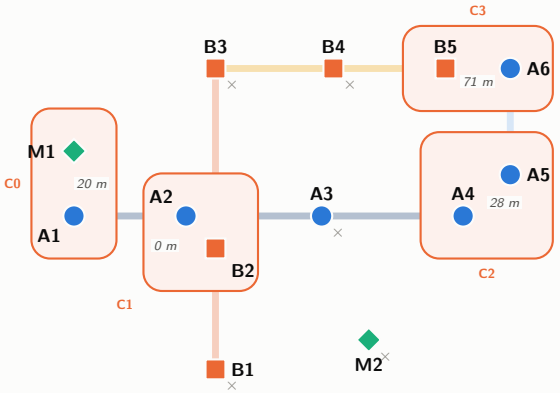
18 intercambios (DBSCAN) **19** transfers.txt

20 match con nodos OSM **21** lado de la calle **22** huecos de cobertura

Ninguno de estos pasos agrega nodos, aristas o pesos al grafo de la Fase 2.

PASO 18 Intercambios con DBSCAN

eps = 120 m, min_samples = 2: basta un vecino a $\leq$ 120 m y los grupos se encadenan



× = ruido (sin vecino a 120 m): A3, B1, B3, B4, M2

	Miembros	Modos	multi	físicas
C0	A1, M1	tranvía	×	2
C1	A2, B2	tranvía, bus	✓	1
C2	A4, A5	tranvía	×	2
C3	A6, B5	tranvía, bus	✓	2

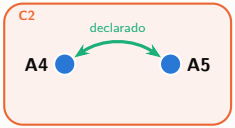
1. DBSCAN en UTM sobre las 13 paradas
2. Se unen con grupos `parent_station` (A4, A5) y de dedupe (A2, B2)
3. Componentes con $\geq$ 2 paradas, numerados por el `stop_uid` menor
4. M1 no aporta modo: MiBici no tiene `route_type`

C1: `physical_stop_count = 1`

Dos registros de un mismo lugar. Es multimodal, pero no hay que caminar: un "fantasma".

PASO 19 Contraste con transfers.txt

¿Los transbordos declarados caen dentro de un mismo clúster?



✓ declared_co_clustered



× declared_not_co_clustered

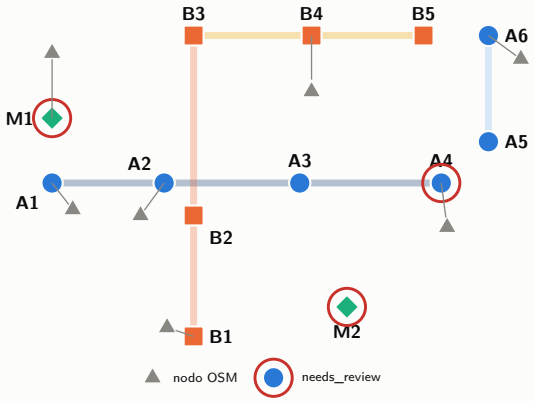
Métrica del reporte	Valor
declared_co_clustered	1
declared_not_co_clustered	1 (con ejemplo)
clusters_with_declared	1 (C2)
clusters_without_declared	3 (C0, C1, C3)

- B4–B5 está a 200 m: el operador declara un transbordo que DBSCAN (120 m) no ve
- C3 es un intercambio real (tren + bus) que ningún feed declara
- Resultado: `interchanges_report.json/.md` y tabla `interchanges`

PASO 20

Match de paradas con nodos OSM

Vecino más cercano por KNN entre nodos con etiquetas de transporte público



Parada	nodo OSM	d (m)	tipo	conf.
B1	bus_stop	5	1.0	0.90
B4	bus_stop	5	1.0	0.90
A1	tram_stop	7	1.0	0.86
A2, B2	tram_stop	8	1.0	0.84
A6	tram_stop	10	1.0	0.80
A4	station	20	0.8	0.48
M1	bus_stop	60	1.0	0.00
M2	ninguno cerca	–	–	0.00

Fórmula

$conf = \max(0,\ 1 - d/50) \times tipo$

tipo: andén/bus_stop/tram_stop 1.0 · station 0.8 (parada normal)

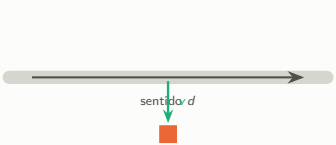
`needs_review` si no hay candidato, $d > 50$ m o $conf < 0.5$

MiBici siempre cae en revisión: el matcher no busca `amenity=bicycle_rental`.

PASO 21 Lado de la calle

Signo del producto cruz respecto al sentido de digitalización de la calle OSM más cercana

B1 derecha · confianza alta



cruz < 0

B4 izquierda · confianza baja



pie a 7 m de la esquina (< 10 m)

A3 on · confianza baja



$|v| \leq 1$ m: andén central

Regla

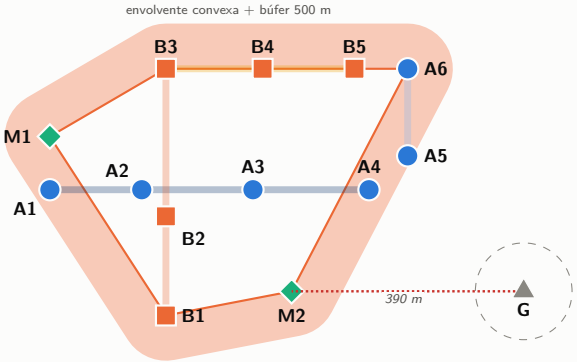
d = segmento más cercano (invertido si `oneway = -1`)
v = parada — pie perpendicular $\quad cruz = d_x v_y - d_y v_x$
> 0 izquierda · < 0 derecha · $|v| \leq 1$ m on

Uso y límites

Sale en `stop_side_of_street`. En calles de doble sentido el lado es relativo a la digitalización, no al sentido del autobús. No toca el grafo.

PASO 22 Huecos de cobertura

La pregunta inversa: ¿qué transporte conoce OSM que los feeds no tienen?



Algoritmo

1. *área* $=$ búfer(envolvente(13 paradas), 500 m)
2. Cada nodo OSM de transporte dentro del área
3. Parada más cercana por KNN (geodésica)
4. Hueco si esa parada está a $>$ 100 m

Nodo	tipo	más cercana	m
G	bus_stop	M2	390

Resultado: `osm_coverage_gaps.gpkg` con 1 punto. Es una lista para revisar: hueco en los feeds o ruido en OSM.

Resumen: qué toca el grafo

En el modelo: 13 nodos y 16 arcos en `station_network`

Paso	Nodos	in_vehicle	walk	same_station	Efecto en el modelo
6 fecha de servicio		✓			R30 desaparece
7 frequencies		×			6 corridas sólo para headways
8 headways		×			CSV y tablas; sin costo de espera
9 route–stop links		✓			8 arcos con minutos
10 proximidad			✓		4 pares candidatos
11 dedupe entre feeds			✓	✓	A2≡B2: 1 par sale de walk, entra 1 same_station
12 conectividad de rutas					otro grafo (rutas)
13 caminata OSM			✓ (pesos)		A6–B5: 1 → 2 min
14–15 listas y calidad					etiquetas, sin cambios
16 red de estaciones	✓	✓	✓	✓	ensambla todo
18–19 DBSCAN, transfers					4 clústeres (tabla aparte)
20–22 OSM					confianza, lado, 1 hueco (aparte)

El grafo es

paradas de Fase 1 + orden programado del día + pares de proximidad limpiados por dedupe + (opcional) minutos de caminata por calles

Fuera del grafo

frecuencias y headways · intercambios DBSCAN · match OSM · lado de la calle · huecos de cobertura